

Problem J1. Concert Tickets

Besa can buy the desired tickets exactly when the already-sold tickets plus the desired tickets do not exceed the total number of tickets. If $B + P > T$, print N. Otherwise, print Y and the number of tickets remaining,

$$T - P - B.$$

Complexity. This is an $O(1)$ calculation.

Problem J2. Olympic Scores

Read the five judges' scores and the difficulty factor. The highest and lowest judge scores are discarded, so if S is the sum of all five scores, H is the maximum score, and L is the minimum score, the final score is

$$(S - H - L)D.$$

The scores can be stored in a list, but it is also enough to maintain the running sum, current minimum, and current maximum while reading.

Complexity. The number of scores is fixed, so the running time and memory usage are $O(1)$.

Problem J3. Creative Candy Consumption

At each step, look only at the next uneaten candy of each person and apply the rule from the first subtask to decide whether Ngoc eats, Minh eats, or both eat. Repeating this rule simulates the whole process.

For larger inputs, avoid physically deleting the first character of a string, because repeated deletion can be slow. Instead, keep two indices pointing to the next uneaten candy in each string. Candies to the left of an index are considered consumed even though the original strings are unchanged.

The loop ends when the stopping condition from the statement is reached. Each iteration advances at least one index, so the simulation is efficient.

Complexity. The simulation runs in $O(A + B)$ time, where A and B are the two candy counts, and uses $O(1)$ extra memory.

Problem J4. Snail Path

The snail enters many grid squares, and a square is slimy exactly when it has been entered before. Therefore we need to count revisits.

For small subtasks, a fixed two-dimensional grid is enough. Mark a square when the snail first enters it, and increment the answer if the snail enters a square already marked. For larger movements, the coordinates can become too large for a fixed grid, so store only the set of visited coordinates. A hash set supports fast membership tests and insertions.

An equivalent approach is to store the full sequence of visited squares, then compute total visits minus the number of distinct squares.

Complexity. With a hash set, the running time is linear in the number of steps the snail takes, and the memory usage is linear in the number of distinct squares visited.

Problem J5. Beams of Light

Each light illuminates one interval of parking spots. For a light at position P with spread S , the interval is

$$[\max(1, P - S), \min(N, P + S)].$$

A query asks whether a given spot is covered by at least one interval.

For the largest constraints, use a difference array. For each interval $[a, b]$, add 1 at a and subtract 1 at $b + 1$. A prefix sum over this array gives the number of lights covering each spot; a spot is illuminated exactly when this value is positive. Then each query is answered in $O(1)$.

Another valid approach is to sort and merge all illuminated intervals, then binary search or scan to answer queries.

Complexity. The difference-array solution runs in $O(N + L + Q)$ time and uses $O(N)$ memory.